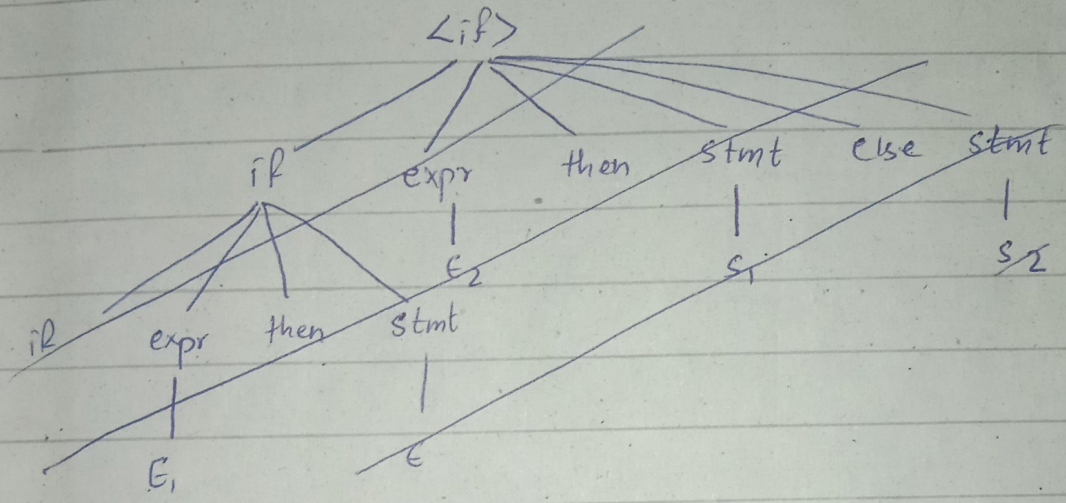
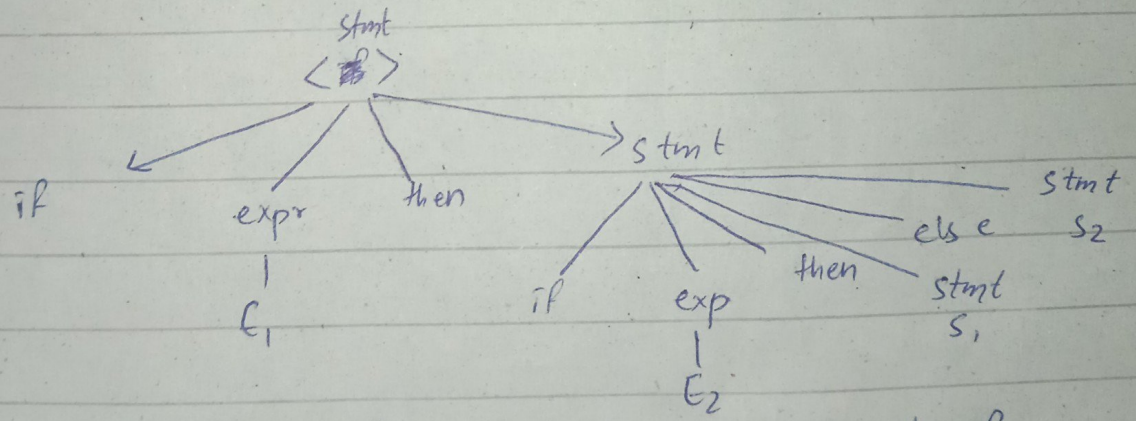
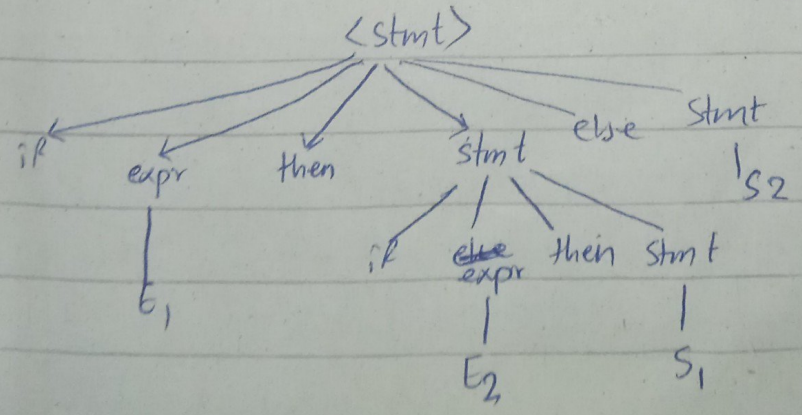
<stmt> → if expr then stmt |
if expr then stmt else stmt | other

statement:
if E_1 then if E_2 then S_1 else S_2





Dangling if



<stmt> → if expr then stmt <else-part> | other

<else-part> → else stmt | ∈

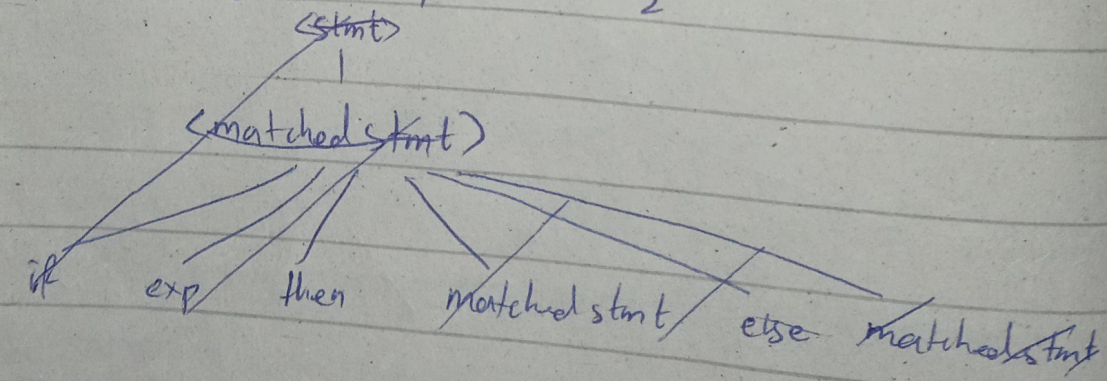
x ——— x

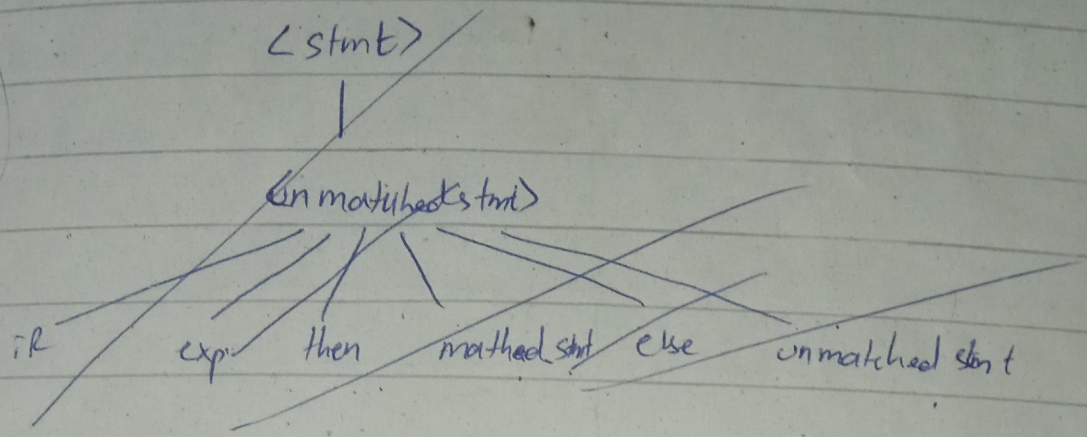
stmt → matched_stmt | unmatched_stmt

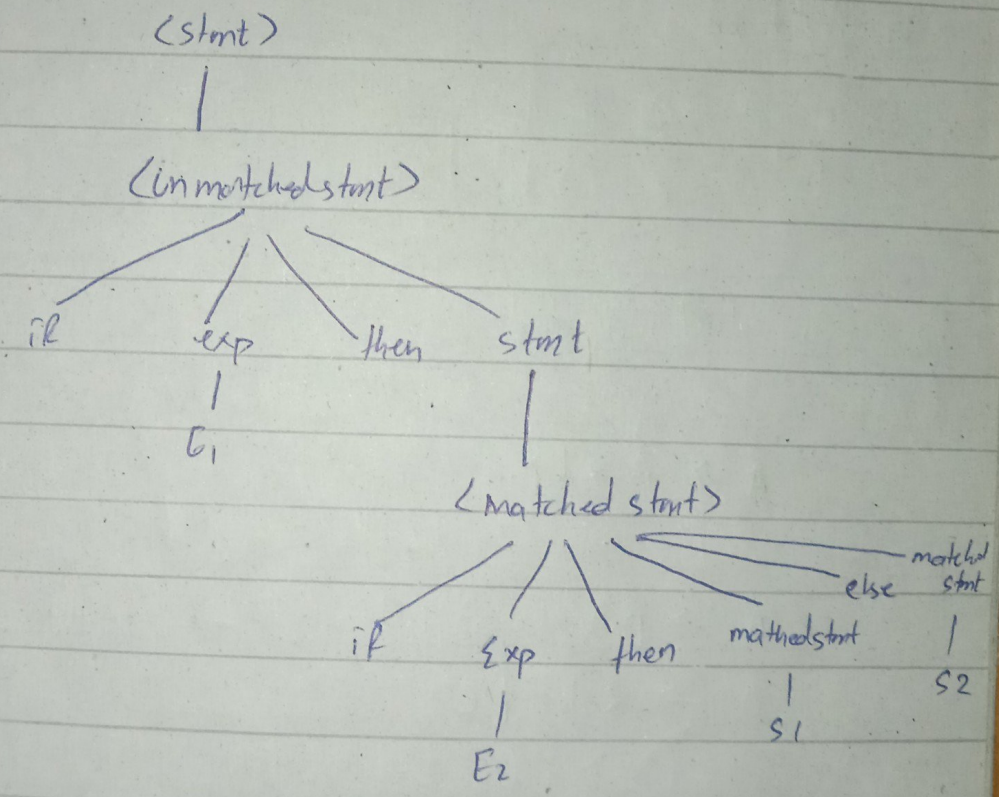
matchedstmt → if exp then matched_stmt else
matched_stmt | other

unmatchedstmt → if exp then stmt |
if exp then matched stmt else unmatchedstmt

if E_1 then if E_2 then S_1 else S_2




<stmt>
<unmatchedstmt>
if
exp
then
matchedstmt
else
unmatchedstmt


<stmt>
<unmatchedstmt>
if
exp
then
stmt
E1
<matchedstmt>
if
Exp
then
matchedstmt
else
matched stmt
E2
S1
S2

Top down parsing

starting non terminal

↓

String

Bottom-up parsing

String

↓

starting non terminal

left recursion

$A \rightarrow Aaba$

To remove left recursion

$A \rightarrow A\alpha \mid \beta$

$\alpha \rightarrow$ non recursive part of recursive rule

$\beta \rightarrow$ non recursive rule

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

Q) $E \rightarrow E+T \mid T$

$T \rightarrow T*F \mid F$

$F \rightarrow (E) \mid id$

$E \rightarrow \underbrace{E}_{A} \underbrace{+T}_{\alpha} \mid \underbrace{T}_{\beta}$

$T \rightarrow \underbrace{T}_{A} \underbrace{*F}_{\alpha} \mid \underbrace{F}_{\beta}$

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

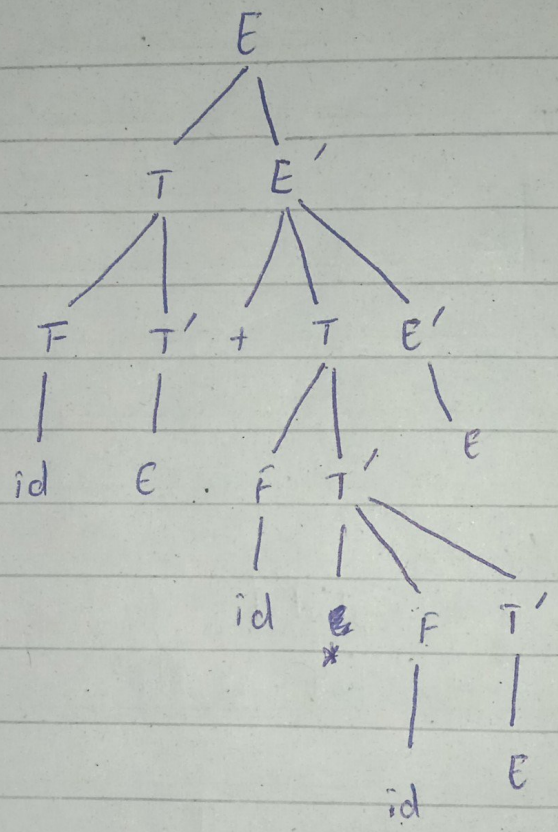
$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

(Q) a + b * c

id + id * id



$A \rightarrow \underbrace{Aab}_{\alpha_1} \mid \underbrace{Acd}_{\alpha_2} \mid \underbrace{Ade}_{\alpha_3} \mid \beta$

$A \rightarrow \beta A'$

$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \alpha_3 A' \mid \epsilon$

$A \rightarrow A\alpha \mid B_1 \mid B_2 \mid B_3$

$A \rightarrow B_1 A' \mid B_2 A' \mid B_3 A'$

$A' \rightarrow \alpha A' \mid \epsilon$

$S \rightarrow (L) \mid a$

$L \rightarrow L, S \mid S$

$S \rightarrow (L) \mid a$

$L \longrightarrow S L'$

$L' \longrightarrow , S L' \mid \epsilon$

Left factoring

stmt → if expr then stmt |
if expr then stmt else stmt

stmt → iEtS | iEtSeS | a

↓

stmt → iEtSS'

s' → eS | ε

Recursive Desant ~~Parser~~ Parcer

Top down

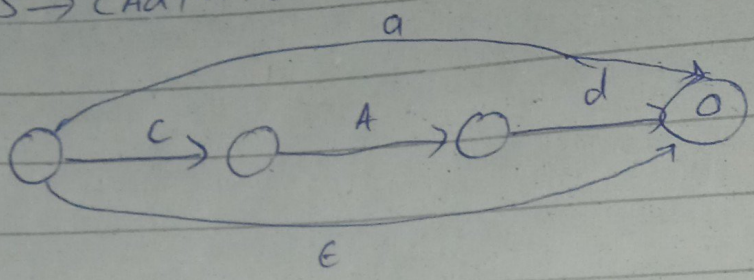
Bottom up

$S \rightarrow cAd$

$A \rightarrow ab \mid a$

Syntax analysis transition diagram

$S \rightarrow cAd \mid a \mid \epsilon$



Q) $E \rightarrow TE'$

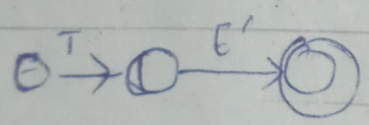
$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

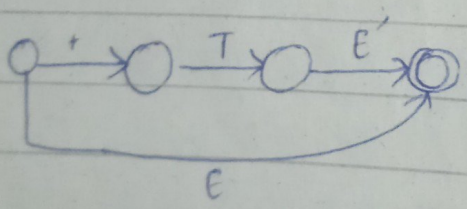
$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

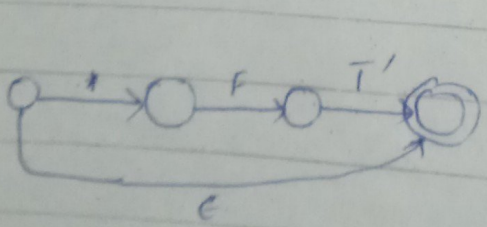
E



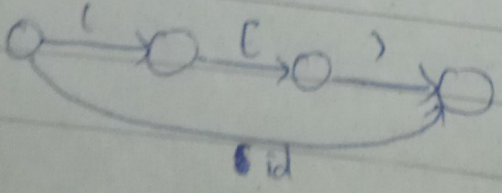
E'



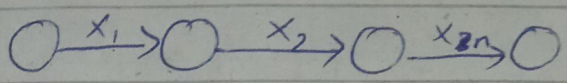
T'



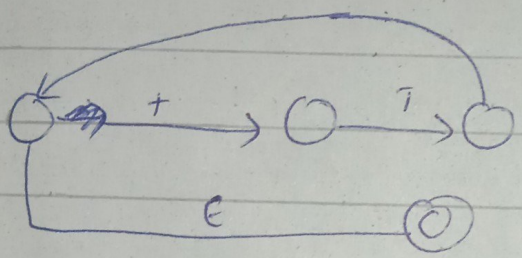
F

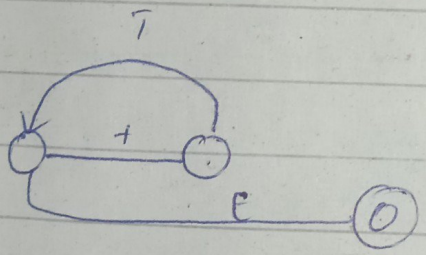


Ex $A \rightarrow x_1 x_2 \ldots x_n$ (from book)

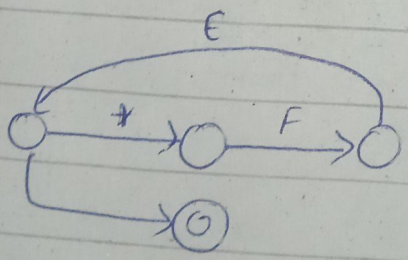


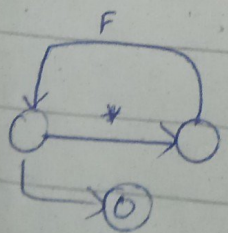
E' ~~can be written~~ can be drawn as



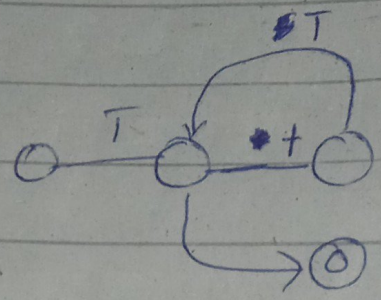


T'

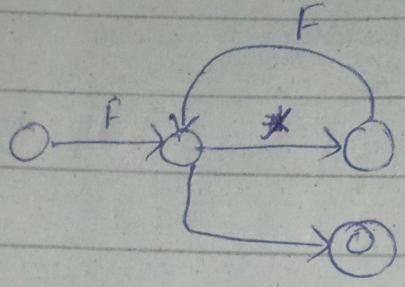




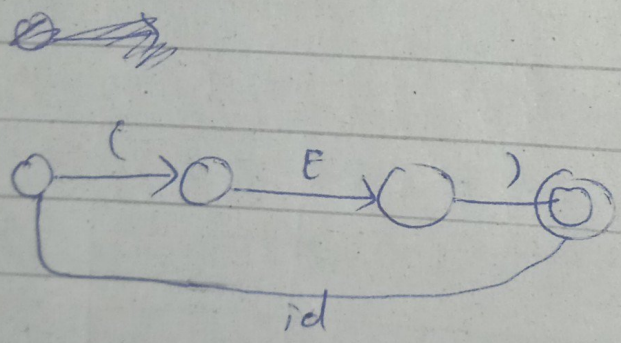
E



T



F



Ipt Symbol

NON Tamina!	id	+	*	(	)	$
E	E → TE′			E → TE′		
E′		E′ → +TE′			E′ → ϵ	E′ → ϵ
T	T → FT′			T → FT′		
T′		T′ → ϵ	T′ → *FT′		T′ → ϵ	T′ → ϵ
F	F → id			F → (E)		

Predictive Parsing Table

Stack	Input	Output
$E	id + id * id $	
$E′T	id + id * id$	E → TE′
$E′T′F	id + id * id$	T → FT′
$E′T′id	id + id * id$	F → id
$E′T′	+ id * id$	T′ → ϵ
$E′T+	+ id * id$	E′ → + TE′
$E′T	id * id$	T → FT′
$E′T′F	id * id$	F → id
$E′T′id	id * id$	
$E′T′F*	* id$	T′ → * FT′
$E′T′F	id $	F → id
$E′T′id	id $	
$E′	$	T′ → ϵ
$	$	E′ → ϵ

Top-bottom parsing

$E \rightarrow TE'$

$E' \rightarrow +TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow *FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

First

First (F) → { (, id }

First (E) → { (, id }

First (T) → {

First (E) = First (T) = First (F) = { (, id }

First (E') = { +, ε }

First (T') → { *, ε }

Follow

Follow (E) = {), $ }

Follow (T) = { +,), $ }

follow (F) = { *, +,), $ }

Follow (E') = { $,) }

Follow (T') = Follow(T) → { +,), $ }

nothing following T

First

Follow

First ter

Add do

If a

so the

first of

) If

its

LL(1)

	id
F	E
E'	
T	
T'	
F	

First

→ First of any non terminal is the first terminal that is read from that non terminal

→ E

Follow

- ~~First terminal that~~

1) Add dollar sign is the starting non terminal
2) If a non terminal is following any non terminal A so the follow of that non terminal would the first of the following A non terminal

3) If first of B includes ∈ then the follow of its originator will be included too.

LL(1)

	id	+	*	(	)	$
E	E→TE'					
E'		E'→+TE'				
T						
T'						
F						

$S \rightarrow iEtSS' \mid a$

$S' \rightarrow es \mid \epsilon$

$E \rightarrow b$

First (S) $\rightarrow \{i, a\}$

First (S') $\rightarrow \{e, \epsilon\}$

First (E$) $\rightarrow \{b\}$

Follow (S) $\rightarrow \{\$, e\}$

Follow (S') $\rightarrow \{e, \$\}$

Follow (E) $\rightarrow \{t\}$

not LL(1)

	i	t	a	e	b	$
S	S→iEtSS'		S→a			
S'				S'→E S'→es		S'→ε
E					E→b	

ambiguity

LL(k)

LL(1)

↓

left to right scanning

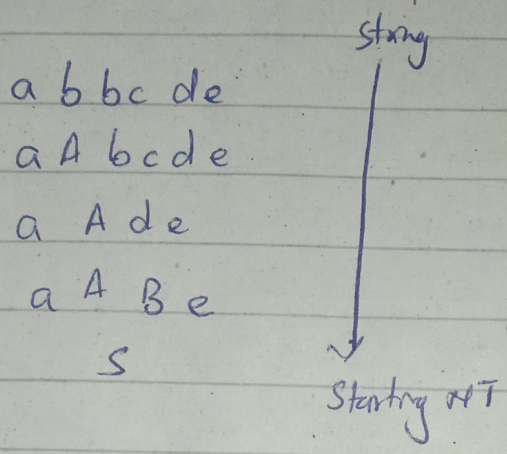
→ left most derivation.

For a LL(1)

↳ unambiguous grammer

Bottom UP Parsing is the reverse of RM&D

$S \rightarrow aABe$
$A \rightarrow Abc \mid b$
$B \rightarrow d$



Right most Derivation

$S \rightarrow aABe$
$aAde$
~~$a\,b$~~
$aAbcde$
$abbcde$

Handler

$A \rightarrow b$
$B \rightarrow d$

Bottom up

↓

Shift-reduce ~~parse~~ parse

↓

LR Parse

↓

LR(k)

→ Canonical

$E' \to .E$
$E \to .E+T$
$E \to .T$
$E \to .T \times F$
$T \to .F$
$F \to .(E)$
$F \to .id$

Goto (I_0, E)

$$\left[\begin{array}{l} E' \to E. \\ E \to E.+T \end{array}\right] I_1$$

goto (I_0, T)

$$\left[\begin{array}{l} E \to T. \\ T \to T.*F \end{array}\right] I_2$$

goto $[I_0, F)$

$$[T \to F.]\ I_3$$

goto $(I_0, ($

$F \to (.E)$

$E \to .E+T$

goto $(I_0, ()$

$$\begin{bmatrix} F \to (.E) \\ E \to .E+T \\ E \to .T \\ E \to .T*F \\ T \to .F \\ F \to .(E) \\ F \to .id \end{bmatrix} I_4$$

goto (E_0, id)

$[F \to id.] \; I_5$

goto $(I_1, +)$

$$\begin{bmatrix} E \to E+.T \\ T \to .T*F \\ T \to .F \\ F \to .(id) \\ F \to id \end{bmatrix} I_6$$

goto (I_4, E)

$$\begin{bmatrix} F \to (E.) \\ E \to E.+T \end{bmatrix} I_8$$

goto $[I_2, *)$

$$\begin{bmatrix} T \to T*.F \\ F \to .(E) \\ F \to .id \end{bmatrix} I_7$$

goto (I_4, T)

$$\begin{bmatrix} E \to T. \\ T \to T. * F \end{bmatrix} I_2$$

goto (I_4, F)

$$[T \to F.]\ I_3$$

goto (I_4, id)

$$[F \to id.]\ I_5$$

goto (I_7, F)

$$[T \to T * F.]\ I_{10}$$

goto (I_6, T)

$$\begin{bmatrix} E \to E + T. \\ T \to T. * F \end{bmatrix} I_9$$

goto $(I_8,))$

$$[F \to (E).]\ I_{11}$$

goto $(I_4, +)$

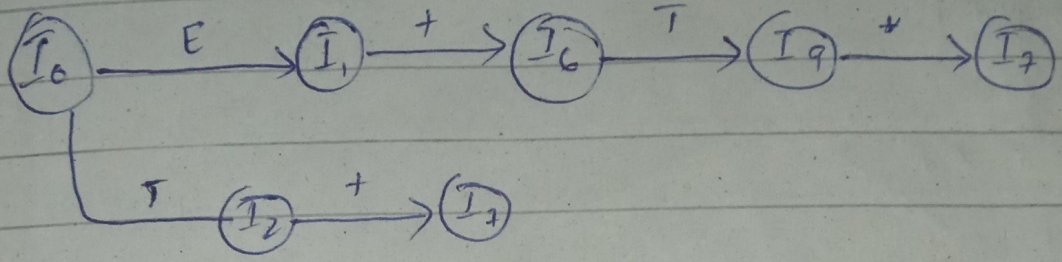
$$\begin{bmatrix} E \to E + .T \\ T \to .T * F \\ T \to .F \\ F \to .(E) \\ F \to .id \end{bmatrix} I_6$$

goto $(I_9, *)$

$$\begin{bmatrix} T \to T * .F \\ F \to .(E) \\ F \to .id \end{bmatrix} I_7$$

I_0

T


I_0
E
I_1
+
I_6
T
I_9
*
I_7
T
I_2
+
I_3

LSR

id +

① $E \rightarrow E+T$

② $E \rightarrow T$

③ $T \rightarrow T * F$

④ $T \rightarrow F$

⑤ $F \rightarrow (E)$

⑥ $F \rightarrow id$

First (E) → ((, id)

First (T) → ((, id)

First (F) → ((, id)

Follow (E) → ~~(+, $)~~ (+,), $)

Follow (T) → ~~(*, ( ) (*, (, id)~~ (*, +,), $)

Follow (F) → (*, +,) $)

terminal → ~~action~~ goto

non terminal → action.

S5 → Shift and move to State 5

r → reduce → states completed

LSR

id + * ( )

Child to parent → synthesized (~~synthesis~~) — S-attributed

Parent to child → inherhited

Semantic Analysis

Syntax Directed Translation

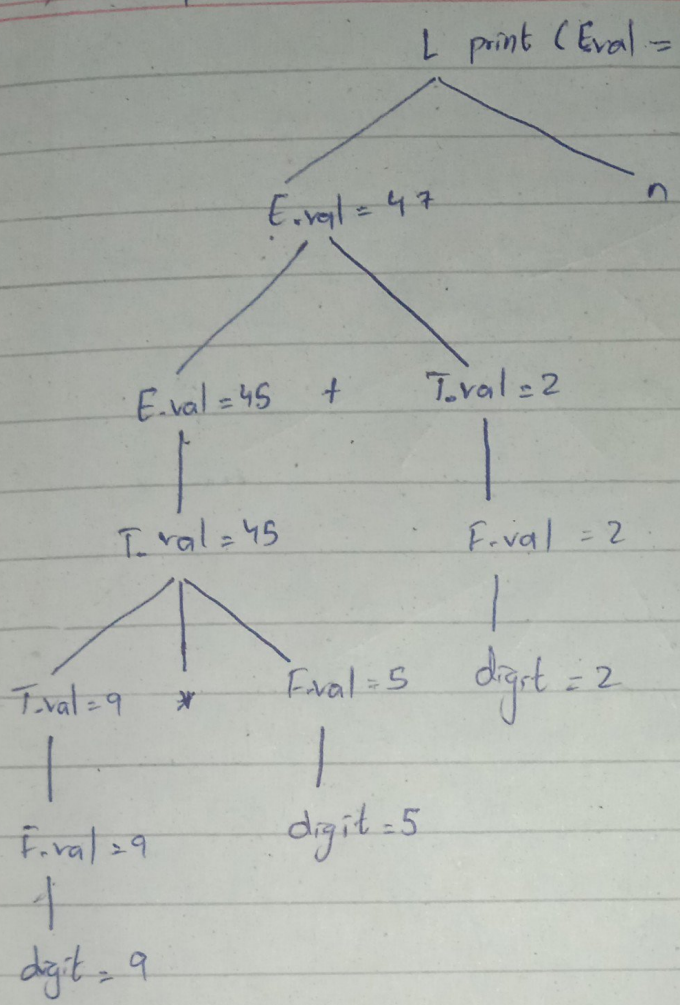
- Syntax Directed Definition
- Translation scheme

Input → parser → SDT → evaluation order

Production	Semantic rule
$L \to E_n$	Print (E.val)
$E \to E+T$	$E.val = E_1.val + T.val$
$E \to T$	$E.val = T.val$
$T \to T*F$	$T.val = T_1.val$
$T \to F$	$T.val = F.val$
$F \to (E)$	$F.val = E.val$
$F \to digit$	$F.val = digit.lexval$

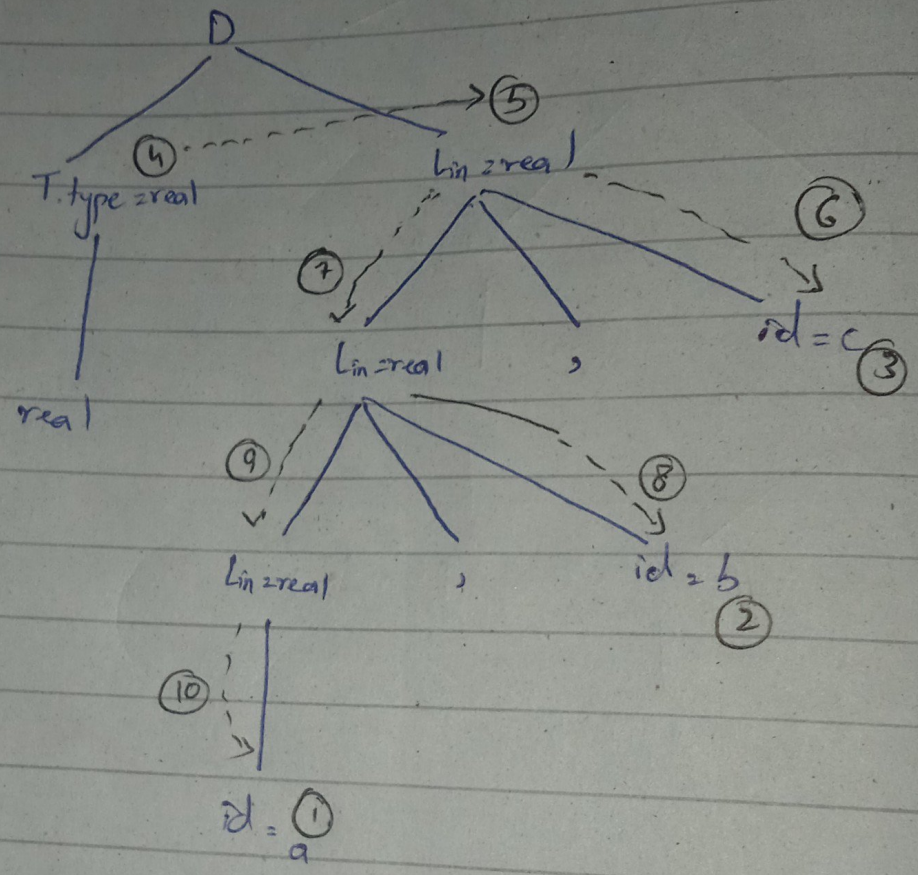
→ annotated parse tree

9*5+2



Product	Semantic Rule
$D \rightarrow TL$	$L_{in} = T_{type}$
$T \rightarrow int$	T.type = integer
$T \rightarrow real$	T.type = real
$L \rightarrow L_{in}$, id	$L_1 = L_{in}$ addtype (identifier)
$L \rightarrow id$	addtype (identifier)

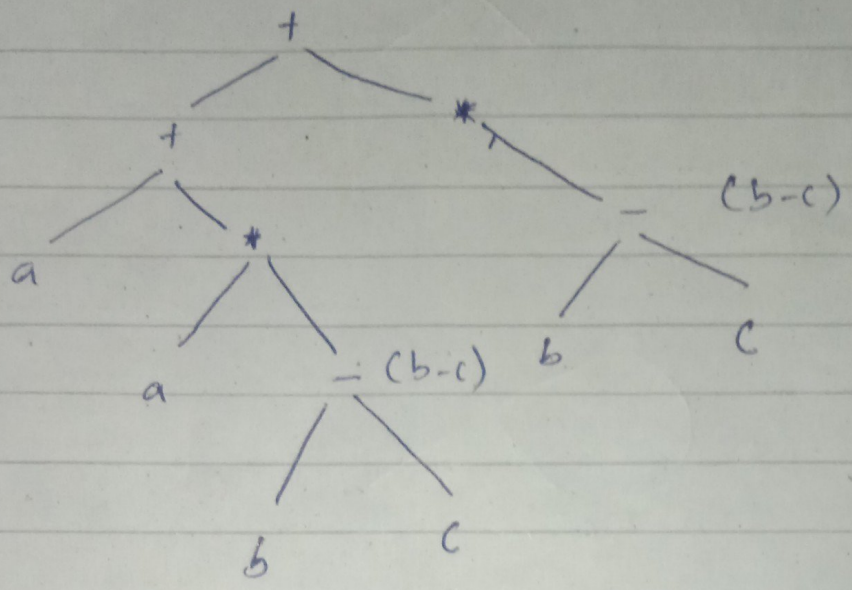
real a, b, c;



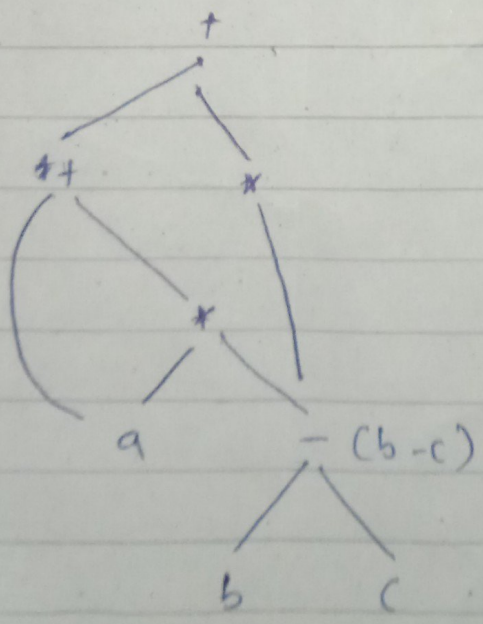
- inherited values
- evaluation order
- Dependency Graph

Acyclic Graph

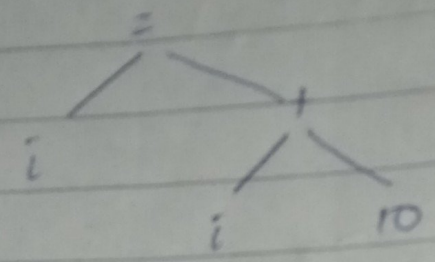
a + a * (b - c) + d * (b - c)



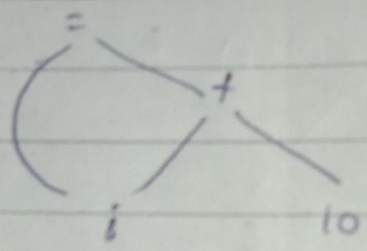
Result of asyclic graph



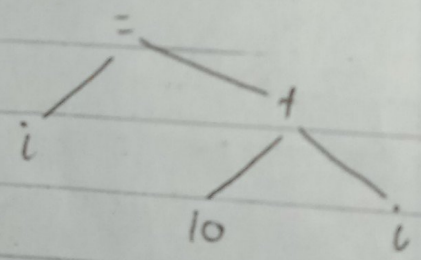
i=i+10

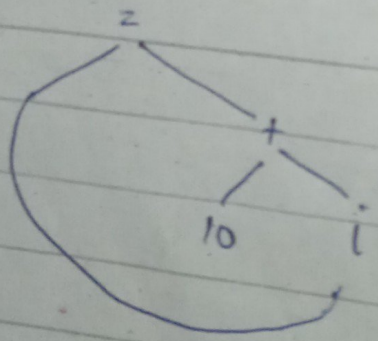


asyclic Graph



→



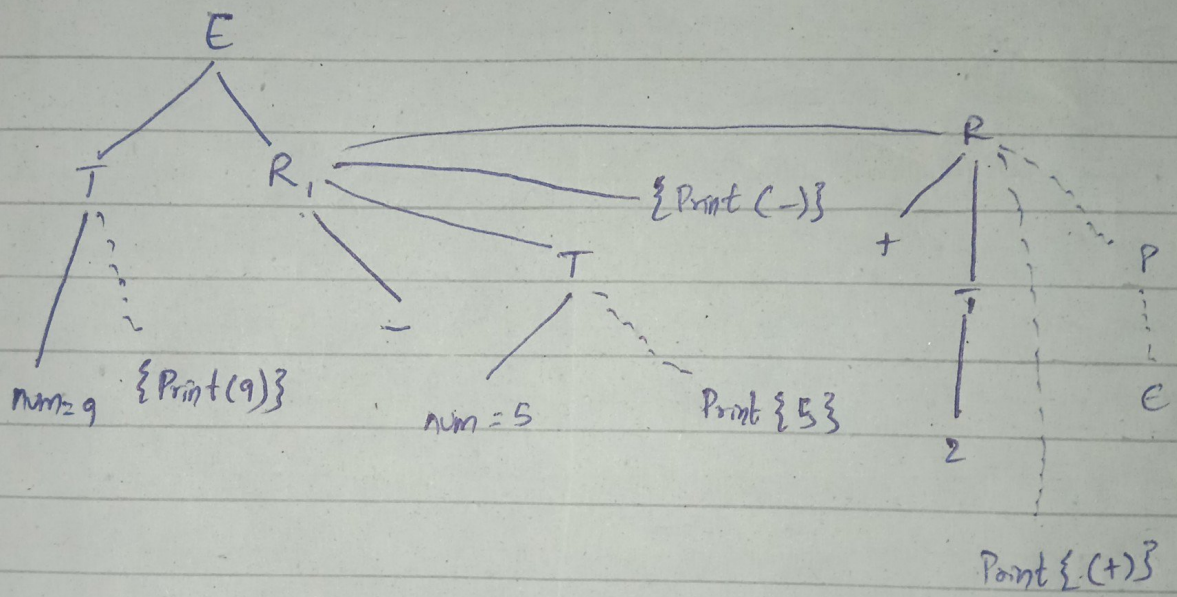


Translation scheme

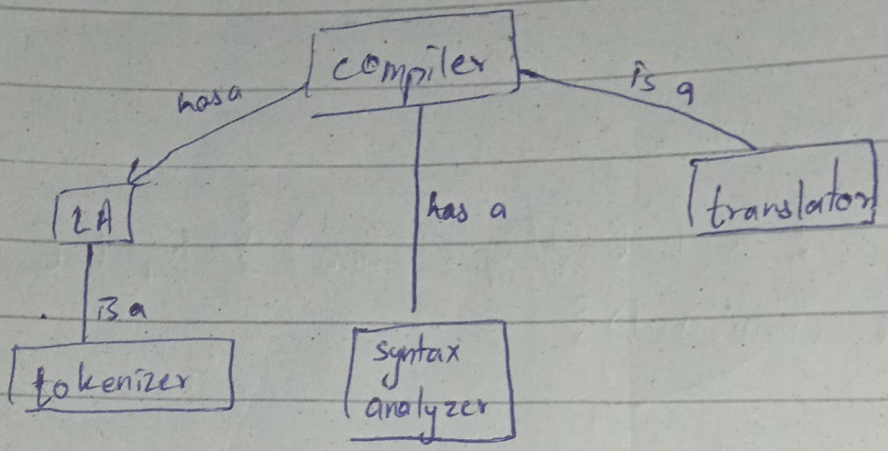
$E \rightarrow TR$

$R \rightarrow addop\ T\ \{Print(addop.lexeme)\}\ R_1 \mid \epsilon$

$T \rightarrow num\ \{Print(num.val)\}$



is a has a



Introductn

Parser:

given